A cheaper genus-2 real-model addition for the H2 glued index calculus

Status. Engineering result, measured and verified. Protocol frozen before the runs: `PROTOCOL.md` (sha256 `3c604cd3…9597f`, recorded in every run's `env.txt`). Every answer verified (`k·P = Q`): 400 targets × 3 arms (1,200 rows) in the paired runs, all verified. Tool: `examples/n2_routes.rs`, modes `glue2check`, `glue2` (new) with the existing modes replaying byte for byte.

Answer

1. **New formulas, derived and implemented here** (`radd2_pre`, `rdbl2_pre`, `rpost2`; no literature count is used). Counted per operation, identical at every size from 20 to 44 bits:

operation	baseline `radd_fast`	new `radd_fast2`	change
addition (generic pair)	41 M + 1 I	**30 M + 1 I**	−11 M (−27%)
walk addition `D + R_j`, `R_j` fixed ("mixed")	41 M + 1 I	**30 M + 1 I**	same as addition: no gain from fixing `R_j`
doubling	51 M + 1 I	**39 M + 1 I**	−12 M
measured per Jacobian op in the arm (setup doublings included), 40 bits	41.2 M + 1.00 I	**30.2 M + 1.00 I**	−11.0 M

   Of the 11 M saved on an addition, **2 M are duplicated products** that the old code evaluated twice (`c3·c2` and `c3·c1`; removing them alone gives 39 M + 1 I, measured on a scratch copy). They are an accounting artefact, not an algorithmic gain, and are separated from the **9 M structural saving**.

2. **Verification.** `radd_fast2 = radd` (the generic law) on **every one of 175,000 pairs on glued instances** (10,000 additions, 5,000 doublings and 20,000 negation/zero/repeated-operand pairs at each of the sizes 20, 28, 36, 40, 44 bits), on **every ordered pair of reduced divisors** of random sextics over `F_7`, `F_11`, `F_13` (**142,636 pairs** on nine monic sextics, 58,526 on three twisted ones; doublings, equal-u pairs and negations included; table below), and the batched version on all of them. Unit tests, `rustfmt`, `clippy -D warnings` pass.

3. **Replay.** `N2_SIZES=28 N2_HS=1 glue` reproduces the 80 rows of `H2-glue-run-20261018T2240Z/glue.csv` at 28 bits, `h = 1`, in every column except the trailing seconds. The `R` and `Jglue` rows of the new `glue2` run reproduce **all 640** rows of that file the same way.

4. **Pipeline result** (same instances, same seeds, same walks; 40 targets per cell; end-to-end cold cost). At 40 bits, `h = 1`, `I = 3M`, measured Jacobian cost, native work at 0 / 0.1 / 1 GAE:

	validated baseline	new	change
IC / matched rho	1.78 / 1.80 / 1.88	**1.39 / 1.48 / 1.75**	IC cost −21.6% / −17.8% / −6.9%

   Distance to rho remaining at 40 bits, `h = 1`: 1.39×, 1.48×, 1.75×. The 12-cell band is still unresolved at every size (IC cheaper in 3 of 12 cells at 40 bits, as before; in 5 instead of 4 at 28 bits; its upper end falls from 1.88 to 1.75). **No size reaches parity.**

5. **Classification: engineering.** Same algorithm, same walks, cheaper primitive. A stage improvement that moved the end-to-end `S`; it is not an advance, and not a relabelling except for the 2 M noted above.

6. **Where the rest is.** Parity at `w = 0` (native work free) needs about **18.9 M + I per operation at 40 bits** (25.8 at 28 bits), measured from these rows; the formulas have 30 M. At `w = 1` GAE no formula can reach it: each walk step carries about 7.3 native operations. The next improvements are named below.

The formulas

Real model `y² = f(x)`, `f` monic of degree 6 over `F_p`, `D = (u, v)` with `u = x² + a1 x + a0`, `v = b1 x + b0`, the class `[D − D∞]`. The second operand is `u2 = x² + c1 x + c0`, `v2 = d1 x + d0`.

Addition (`D1 ≠ D2`, coprime `u`; 12 M, one inversion, 18 M):

- `z1 = a1 − c1`, `z2 = c0 − a0`, `z3 = a1 z1 + z2` (1 M), `t = a0 z1` (1 M), `r = z2 z3 + z1 t` (2 M). The inverse of `u2` mod `u1` is `(z1 x + z3)/r`.
- With `w = v1 − v2`: because `z3 − a1 z1 = z2`, the unnormalised slope is `s1' = z1 w0 + z2 w1`, `s0' = z3 w0 − t w1` (4 M, no Karatsuba needed).
- `big2 = a0 + c0 + a1 c1` (1 M); `r²`, `dq = (r − s1')(r + s1')`, `w = r·dq` (3 M).
- **One inversion** `iw = 1/w`, giving `1/r = dq·iw`, `1/dq = r·iw`, and `1/(1 − s1²) = r²/dq` (3 M); `s1 = s1'/r`, `s0 = s0'/r` (2 M).
- The cubic `v = v2 + u2 s` by Karatsuba (3 M). The top coefficients of `f − v²` give `n5 = f5 − 2c3c2`, `n4 = f4 − c2² − 2c3c1` (3 M), and `u' = x² + up1 x + up0` with `up1 = n5/q2 − big3`, `up0 = n4/q2 − big2 − up1·big3` (3 M; no `q1`, `q0`, `q2` needed).
- `v' = −(v mod u')` by Horner: `t = c3 up1 − c2`, `x1 = c1 + up1 t − c3 up0`, `x0 = c0 + up0 t` (4 M).

Doubling (39 M + I): `k mod u` from `f mod u²` (10 M), `Res(u, v)` and the slope (8 M), then the same inversion block and tail.

Where the 11 M go (additions): duplicated products −2 (accounting); resultant −1; slope −2; Karatsuba in the cubic −1; `up1, up0` from `n/q2` −3; Horner reduction −2.

Mixed addition. The walk adds fixed `R_j`. No product that involves only the second operand occurs in the pre-inversion phase: `z1, z2, z3, t` and `s1', s0'` mix both operands through differences, and `a1·c1`, `c1·s1`, `c0·s0` mix them again. There is nothing to precompute, so the "mixed" cost equals the general addition (30 M + I, measured: the `walk add` row of `glue2_checks.txt`). This was derived, not assumed.

Batched inversion. The pre-inversion state is split from the post-inversion phase (`radd_batch2`), so `W` independent additions share one inversion (Montgomery). Counted (stage diagnostic, `glue2check`): 32.25 M + 0.25 I at `W = 4`, 32.81 M + 0.062 I at `W = 16`, 32.95 M + 0.016 I at `W = 64`. At `I = 3M` this is **33.0 M-equivalent at `W = 16`, the same as unbatched (30 M + 3 M)**: the I-amortised-to-3M corner of the band is already what batching delivers, so batching moves only the `I = 2·log2 p` corner (an operation falls from about 70 to about 33 there). A single walk is sequential; using this needs `W` lockstep walks, a different (unpaired) algorithm that was not run. It is a stage diagnostic, not a speedup.

Checks (run `N1-check-run-20261024T2146Z`)

check	result

exhaustive F_7, F_11, F_13, three monic sextics each, all ordered pairs (incl. 0, doublings, equal u, negations)	radd_fast2 = radd on every pair (see glue2_checks.txt)
same, one non-monic (twisted) sextic per field	radd_fast2 = radd on every pair; the **old radd_fast agrees on only 399 of 729 pairs** at F_7 (positive control of the detector; see the limitation below)
glued instances, 20 / 28 / 36 / 40 / 44 bits	adds 10,000, doubles 5,000, special 20,000 per size: 35,000 of 35,000 agree at each size, 175,000 of 175,000 in all
batched W = 4, 16, 64	all agree with radd
counted cost	add 30.00 M + 1.00 I; walk add 30.00 M + 1.00 I; double 39.00 M + 1.00 I (small sizes carry rare fall-backs: 30.26 M at 20 bits)

A latent limitation of the baseline, found by the control. radd_fast (and so Arm Jglue) assumes a monic sextic. Glue::new can return a twisted F_p model with leading coefficient κ; there the old fast law is wrong. All 32 instances of H2 (the same 32 in N2) and the 8 of N3 have twist = 0, so no recorded result is affected, but a twisted instance would never terminate in Jglue. The new formulas fall back to radd there.

Measurements

Runs (single thread, N2_THREADS=1, every one with env.txt):

run	what	rows
N0-replay-run-20261024T2146Z	N2_SIZES=28 N2_HS=1 glue; diff against H2	80, identical
N1-check-run-20261024T2146Z	glue2check	175,000 pairs + exhaustive
N2-glue2-run-20261024T2146Z	glue2, 28–40 bits, h = 1, 4, 4 × 10 per cell	960 (320 per arm)
N3-glue2-44bit-run-20261024T2149Z	glue2, 44 bits, h = 1, 4, 4 × 10 per cell	240 (80 per arm)

R and Jglue rows equal H2's (all 640 at 28–40 bits); Jglue2 equals Jglue row by row in steps, relations, Jacobian operations, inversions, native work and verification (400 of 400 paired rows including 44 bits; pairing_check.txt in each run). So steps/p is unchanged (0.435, 0.426, 0.439, 0.443 at 28–40 bits and h = 1; rho 0.956, 0.844, 0.778, 0.860) and **only the cost per step moved**.

h = 1: IC/rho at the gate's three points (I = 3M, measured Jacobian cost; native work at 0 / 0.1 / 1 GAE), baseline then new, and the 12-cell band

bits	baseline Jglue/rho	Jglue2/rho	IC cost change	12-cell band, baseline	12-cell band, new
28	1.50 / 1.52 / 1.61	1.17 / 1.23 / 1.47	−21.9% / −18.9% / −8.4%	0.594–1.611, IC cheaper in 4/12 (unresolved)	0.594–1.475, IC cheaper in 5/12 (unresolved)
32	1.69 / 1.72 / 1.82	1.32 / 1.41 / 1.68	−21.7% / −18.2% / −7.4%	0.706–1.818, IC cheaper in 3/12 (unresolved)	0.706–1.684, IC cheaper in 3/12 (unresolved)
36	1.92 / 1.95 / 2.04	1.50 / 1.60 / 1.89	−21.6% / −17.9% / −7.1%	0.815–2.035, IC cheaper in 1/12 (unresolved)	0.815–1.892, IC cheaper in 1/12 (unresolved)
40	1.78 / 1.80 / 1.88	1.39 / 1.48 / 1.75	−21.6% / −17.8% / −6.9%	0.759–1.876, IC cheaper in 3/12 (unresolved)	0.759–1.745, IC cheaper in 3/12 (unresolved)
44	1.72 / 1.74 / 1.81	1.35 / 1.43 / 1.69	−21.6% / −17.8% / −6.9%	0.735–1.812, IC cheaper in 3/12 (unresolved)	0.735–1.687, IC cheaper in 3/12 (unresolved)

h = 4: IC/rho at the gate's three points (I = 3M, measured Jacobian cost; native work at 0 / 0.1 / 1 GAE), baseline then new, and the 12-cell band

bits	baseline Jglue/rho	Jglue2/rho	IC cost change	12-cell band, baseline	12-cell band, new
28	2.09 / 2.16 / 2.44	1.63 / 1.74 / 2.22	−21.9% / −19.1% / −9.0%	0.823–2.440, IC cheaper in 1/12 (unresolved)	0.823–2.221, IC cheaper in 1/12 (unresolved)
32	2.34 / 2.38 / 2.54	1.83 / 1.95 / 2.34	−21.7% / −18.4% / −7.8%	0.972–2.536, IC cheaper in 1/12 (unresolved)	0.972–2.338, IC cheaper in 1/12 (unresolved)
36	2.77 / 2.80 / 2.88	2.17 / 2.29 / 2.67	−21.6% / −18.1% / −7.4%	1.176–2.878, IC cheaper in 0/12 (rho cheaper in every cell)	1.176–2.665, IC cheaper in 0/12 (rho cheaper in every cell)
40	2.68 / 2.69 / 2.73	2.10 / 2.21 / 2.53	−21.6% / −18.0% / −7.3%	1.144–2.733, IC cheaper in 0/12 (rho cheaper in every cell)	1.144–2.535, IC cheaper in 0/12 (rho cheaper in every cell)
44	2.71 / 2.72 / 2.73	2.13 / 2.23 / 2.53	−21.6% / −18.0% / −7.3%	1.159–2.732, IC cheaper in 0/12 (rho cheaper in every cell)	1.159–2.534, IC cheaper in 0/12 (rho cheaper in every cell)

Notes. "IC cost change" is Jglue2/Jglue − 1 at the same point. The band is the §5a 12-cell band (I ∈ {3M, 2·log2 p M} × Jacobian priced as measured or as one E(F_{p²}) addition × native work at 0, 0.1, 1 GAE, the split test at one I per step); two arms are ranked only where the order holds in all 12 cells, so every row is "unresolved" except those where rho wins in all 12. The 44-bit rows have no H2 baseline; their baseline is the Jglue row of the same run.

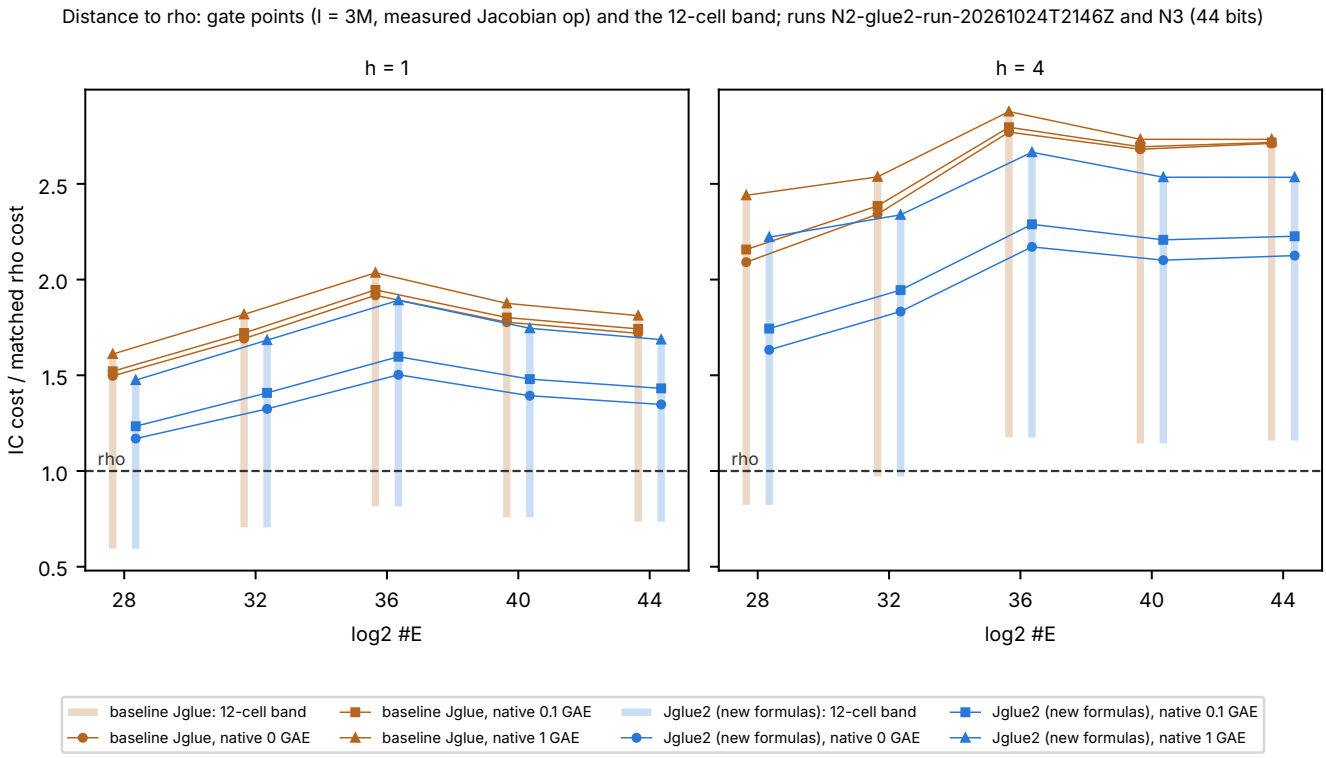


Figure 1: IC/rho

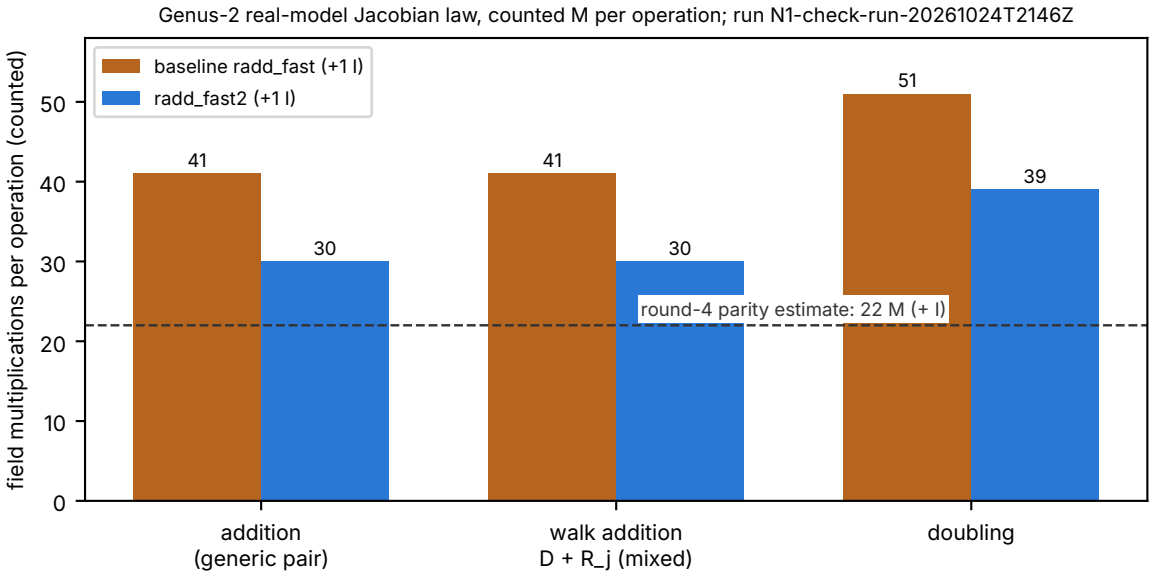


Figure 2: per-op

Distance to rho

- `h = 1`: 1.17 / 1.23 / 1.47 (28 bits), 1.32 / 1.41 / 1.68 (32), 1.50 / 1.60 / 1.89 (36), 1.39 / 1.48 / 1.75 (40), 1.35 / 1.43 / 1.69 (44) at the three gate points.
- `h = 4`: 1.63–2.22 (28), 1.83–2.34 (32), 2.17–2.67 (36), 2.10–2.53 (40), 2.13–2.53 (44); rho is cheaper in every one of the 12 cells at 36, 40 and 44 bits.
- Precomputation rho (Bernstein–Lange), setup excluded: not measured in this study (no such row exists in H2), so not reported.
- Measured break-even at 40 bits, `h = 1`, `I = 3M`, native 0 / 0.1 / 1 GAE: **18.9 M / 13.7 M / unreachable** per Jacobian operation (+ I), against 30.2 M now (and 41.2 M before). The round-4 estimate of 22 M was close to the 28-bit value (25.8 M); at 40 bits the target is lower.

What this does and does not establish

- It establishes a verified, measured improvement of the index-calculus baseline: the same pipeline costs 21.6% less at the gate's first point (`w = 0`), 17.8% at the second and 6.9% at the third, at 40 bits and `h = 1`, and a comparable fraction at the other sizes. Each landed improvement becomes the new baseline: `Jglue2` at 30.2 M + I per operation.
- It does not beat rho, at any size or point. It does not change the `h = 4` verdict (rho cheaper in every cell at 36, 40 and 44 bits).
- Wall-clock seconds are recorded but are not evidence (the repository measures counted operations). They did not change measurably (`Jglue2` 16.7 s against `Jglue` 16.2 s at 40 bits, `h = 1`, a shared host): the cost of this tool is not in the field multiplications.
- The memory result of H4 ($\Theta(p)$ stored relations) is untouched.
- No novelty audit of the formulas was made. Karatsuba-style products, a shared inversion and balanced real-model arithmetic are standard techniques; the count 30 M + I is a measurement of this implementation only.

Next improvements

1. **Native work per step.** About 7.3 native operations per walk step (hash, table lookups, linear-algebra pivots) are priced at up to 1 GAE each; at `w = 1` they exceed the whole formula. A cheaper relation and pivot bookkeeping moves the third gate point, which the formula cannot.

2. **The split test.** The Legendre symbol is priced at one `I` per step (3 M here; 40 M at the other corner). Share an Euclid run with the inversion, or test the discriminant without a symbol.
3. **Another 11 M per operation at `w = 0`** is needed for parity at 40 bits. Candidates: a model with a rational Weierstrass point (not available for prime-order `E`), or fewer Jacobian operations per relation.
4. **Lockstep walks with a shared inversion** for the `I = 2·log2 p` corner; it needs its own protocol amendment and a non-paired measurement.
5. **Setup at small sizes**: the 32 multipliers and restarts are 28% of the operations at 28 bits (ops/step 1.38); a fixed-base table would save about half of that. It changes only the 28–32-bit rows.

Files

`PROTOCOL.md`, `REPORT.md`, `REPORT.pdf`, `run_item.sh`, `build.sh`, `figures/` (`plot.py`, `tables.py`, SVG and PNG), `results/*` (CSVs, `glue2_summary.txt`, `gae_band.txt`, `glue2_checks.txt`, `pairing_check.txt`, `env.txt`); `examples/n2_routes.rs` (additive: `radd2_pre`, `rdbl2_pre`, `rpost2`, `radd_fast2`, `rmul_fast2`, `radd_batch2`, `arm_glue2`, `glue_job2`, `summarize_glue2`, `glue2_checks`, modes `glue2check`, `glue2`, `glue2summarize`; `gae_band` gives the Legendre term to every `Jglue*` arm).